LangGraph Video Scenarios: Detailed Guide

This guide explains the six examples in this folder: the automatic guessing game and five agent workflows from the course video. It describes what the state stores, what each node does, how edges route execution, and what the sample runs produce.

Before the scenarios: how to read a LangGraph

A LangGraph workflow is a directed graph. **State** is the shared data passed from node to node. A **node** is a named Python function that reads the state, does one job, and returns its updates. An **edge** says which node runs next.

The scripts follow this common pattern:

```
graph = StateGraph(MyState)
graph.add_node("work", work_node)
graph.add_edge(START, "work")
graph.add_edge("work", END)
workflow = graph.compile()
result = workflow.invoke(initial_state)
```

`TypedDict` documents the expected state keys and value types for Python and editor tooling. It is not runtime validation. `START` is the graph entry, `END` is a terminal point, and `compile()` turns the graph definition into a workflow that can be invoked.

A plain edge always follows the same path. A conditional edge calls a routing function, which returns a label; a mapping connects that label to a node or to `END`. This is how the examples loop or branch without putting the whole graph inside one large function.

Scenario 1 -- Automatic higher/lower guessing loop

Video: 1:29:25-1:31:05 **File:** `scenario_01_number_guessing_loop.py`

The graph repeatedly guesses an integer from 1 to 20. It starts with the whole range, guesses its midpoint, uses a higher/lower hint to shrink the range, and loops until it wins or reaches its attempt limit.

State

- `player_name`, `greeting`: who is playing and the greeting made by the first

node.

- `secret_number`: the number the guess is checked against. It is included in

the initial state to make this offline example deterministic. The guess node does not read it; the evaluation node acts as the game's answer checker.

- `lower_bound`, `upper_bound`: the inclusive range for the next guess.

- `current_guess`, `guesses`: the latest guess and the full history.

- `attempts`, `max_attempts`: attempt count and the configured cap (default 7).

- `hint`: `higher`, `lower`, or `correct` after evaluation.

- `status`: `playing`, `won`, or `out_of_guesses`.

Nodes and routing

1. `greet_player` creates the greeting from `player_name`.
2. `make_guess` computes `(lower_bound + upper_bound) // 2`, stores it as the

current guess, adds it to `guesses`, and increments `attempts`.

3. `evaluate_guess` compares the guess with the target. If the target is higher,

it raises the lower bound to `guess + 1`; if the target is lower, it lowers the upper bound to `guess - 1`. A correct guess sets `status` to `won`. If the last allowed attempt was wrong, it sets `status` to `out_of_guesses`.

4. `route_after_guess` returns `guess_again` while status is `playing`, or

`finish` otherwise.

```
START  ->  greet_player  ->  make_guess  ->  evaluate_guess
                             ^              |
                             +---- guess_again
                                            +---- finish  ->  END
```

The conditional mapping is `guess_again -> make_guess` and `finish -> END`. Therefore, each wrong guess shrinks the range and repeats; a win or an exhausted attempt limit stops the graph.

Walkthrough of the sample runs

With target `1` and the default 7-attempt cap:

Attempt	Guess	Hint	Remaining range
1	10	lower	1-9
2	5	lower	1-4
3	2	lower	1-1
4	1	correct	1-1

The final state has `guesses` equal to `[10, 5, 2, 1]`, `attempts` equal to 4, and `status` equal to `won`.

With target `20`, the guesses are `[10, 15, 18, 19, 20]`. The first four hints are `higher`, because the target is greater than each guess; the fifth guess wins.

The final sample sets `max_attempts` to 2 and target to `1`. The graph guesses 10 and then 5, then stops with `status` equal to `out_of_guesses`. The lower cap is only to exercise the limit path quickly; normal sample runs use 7.

The midpoint strategy is binary search, so it finds any valid number in this range in at most five guesses. The game remains capped at seven as requested.

Scenario 2 -- Stateless chatbot

Video: 1:31:01 **File:** `scenario_02_stateless_chatbot.py`

This is the smallest chatbot graph: accept one user message, produce a response, and end. The original example calls an LLM. This version uses a deterministic echo so it runs without a model or API key.

State and node

`ChatbotState` has `messages` and `reply`. Each message is a small dictionary with a `role` (`user` or `assistant`) and `content`. `reply_to_latest_message` searches backward for the latest user message, formats a predictable echo, stores it in `reply`, appends an assistant message, then returns the updated state. If there is no user message, it raises `ValueError` rather than silently making an empty reply.

```
START  ->  reply_to_message  ->  END
```

The sample starts with "Hi, I'm Steve. Can you say hello?" The final state has the original user message, the assistant echo in `messages`, and the same text in `reply`. This graph has no conditional branch and does not carry context between separate invocations; that is the difference from Scenario 3.

Scenario 3 -- Chatbot with conversation history

Video: 1:42:54 **File:** `scenario_03_conversation_memory.py`

This scenario demonstrates memory by keeping prior turns in the state and passing the returned state into the next invocation. There is still one graph node, but it can respond using messages from earlier turns.

State, node, and two turns

The state uses `messages` and `reply`, like Scenario 2. The helper `find_name` looks for simple phrases such as "my name is Steve" or "I'm Steve." The node `respond_using_history` finds the latest user message. If it contains a name, the node acknowledges it. If it asks about a name, the node scans earlier user messages for a name and responds from that history. Otherwise, it uses a deterministic fallback echo.

The script runs two turns:

1. Invoke with "Hi, I'm Steve." The result includes the user turn and an

assistant response, "Nice to meet you, Steve."

2. Append "What is my name?" to the returned message history, then invoke the

same workflow again. The node finds Steve in the earlier user message and responds, "Your name is Steve."

```
START  ->  respond_using_history  ->  END
```

The graph itself is straight-line. The "memory" comes from the caller preserving and reusing the returned state between calls, not from a database or a special LangGraph memory feature. The name recognition is intentionally a small regex, not natural-language understanding.

Scenario 4 -- ReAct calculator and tool loop

Video: 2:03:20 **File:** `scenario_04_react_calculator.py`

ReAct means "reasoning and acting": an agent chooses an action, a tool performs it, then the agent decides whether another action is needed. Here, the plan is provided as structured state instead of being chosen by an LLM.

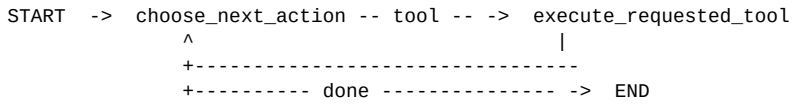
State

- `tool_plan`: the ordered calculator operations.
- `next_tool_index`: which operation should be requested next.
- `pending_call`: the operation currently waiting for the tool node.
- `result`: the latest numeric result.
- `route`: `tool` while an operation remains, or `done` when finished.
- `messages`: a readable trace of requested operations and tool results.

Each `ToolCall` contains a tool name (`add`, `subtract`, or `multiply`) and operands. A `left` operand of `None` means "use the previous result."

Nodes and execution trace

1. `choose_next_action` reads the next operation from `tool_plan`. It sets `pending_call` and route `tool`; after the plan is exhausted, it sets route `done`.
2. The conditional edge sends `tool` to `execute_requested_tool`, or `done` to `END`.
3. `execute_requested_tool` performs the requested arithmetic, stores the result, advances `next_tool_index`, and clears `pending_call`.
4. A plain edge returns from the tool node to `choose_next_action`.



The sample plan is:

1. Add 40 and 12 -> 52.
2. Multiply the previous result (52) by 6 -> 312.
3. The plan is exhausted, so the agent routes to `END`.

The final state has `result` equal to `312` and a trace in `messages`. This example does not interpret a natural-language request or generate the extra conversational response from the video; it focuses on the agent/tool/agent graph pattern.

Scenario 5 -- Drafter with feedback and save

Video: 2:28:46 **File:** `scenario_05_document_drafter.py`

The Drafter scenario models a collaboration loop: apply feedback to a draft, continue while revisions remain, then save when feedback approves the result. The feedback is scripted in the initial state instead of being typed interactively, so the run is repeatable.

State and nodes

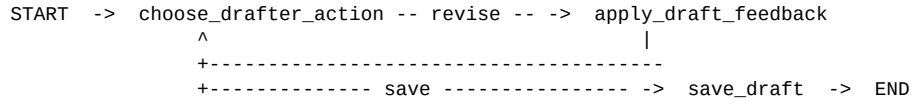
- `draft`: the current document text.
- `feedback`, `feedback_index`: the scripted feedback turns and the next turn

to process.

- `pending_feedback`: the feedback currently being handled.
- `next_action`: `revise` or `save`, selected by the agent node.
- `output_path`, `saved`, `saved_path`: where the result is written and whether

saving completed.

`choose_drafter_action` checks the next feedback. Feedback containing "save," "looks good," or "approved" selects `save`; other feedback selects `revise`. `apply_draft_feedback` appends the requested change to the draft and advances the feedback index. `save_draft` writes the current draft to the provided path and records the saved status in state.



Sample run

The initial draft is "Hello team, our project is progressing." The first feedback asks to add that the first review is due Friday, so the revision node appends that instruction. The next feedback says "Looks good; save this draft," so the graph takes the save path. It writes `draft_output.txt` beside the script, and the final state reports `saved: True` and the path.

This deterministic revision does not rewrite prose like an LLM would; it appends the feedback verbatim to make the state transition visible. The output path is in the state, so the save node does not hard-code a destination.

Scenario 6 -- Local retrieval-augmented Q&A (RAG)

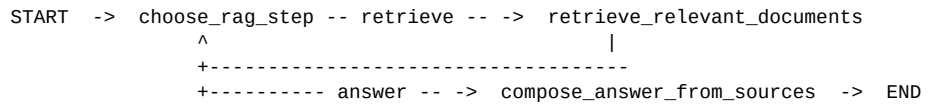
Video: 2:52:09 **File:** `scenario_06_local_rag.py`

RAG stands for retrieval-augmented generation. A typical RAG workflow retrieves relevant source passages, then uses them to answer the question. This offline version keeps that graph structure, but uses local sample text and keyword overlap rather than embeddings, a vector database, or an LLM.

State and nodes

- `question`: the user's question.
- `documents`: local source documents, each with a `title` and `content`.
- `retrieved_documents`: the top matching passages found so far.
- `retrieval_done`: prevents the graph from repeatedly retrieving.
- `next_step`: `retrieve` before searching, `answer` afterward.
- `answer`: the final source-based response or a no-match message.

`choose_rag_step` routes to retrieval until `retrieval_done` is true, then routes to answer composition. `retrieve_relevant_documents` tokenizes the question and documents, removes common words (including the year), ranks by shared terms, keeps up to three matches, and marks retrieval complete. `compose_answer_from_sources` joins the retrieved content or returns a no-match fallback.



Sample inputs and branches

The local corpus contains a market-summary passage stating that the S&P 500 total return for 2024 was 25.02%, plus a technology-sector passage.

- "What was the S&P 500 total return in 2024?" matches the market-summary

passage. The graph returns its text as the answer.

- "What was the rainfall in Sydney in 2024?" has no meaningful word overlap

with either document. The graph returns "No matching passage was found in the local report."

Both runs use the same conditional graph path--retrieve, then answer--but test the found-document and no-document outcomes. Ignoring the year in matching prevents the generic word `2024` from making unrelated documents look relevant.

Keyword overlap is only a teaching substitute. It does not understand synonyms or meaning, rank semantically, or generate a fluent answer. Those are the jobs of embedding search and an LLM in a full RAG system.

Run the examples

From this project directory, run a single scenario like this:

```
uv run scenario_01_number_guessing_loop.py
```

Or run all six scripts:

```
for script in scenario_*.py; do
  uv run "$script"
done
```

Each script prints the final state as a regular Python dictionary and writes a Mermaid PNG diagram beside itself. Rendering diagrams uses the Mermaid image service, so it needs network access; the scripts retry transient failures. Scenario 5 also writes its sample draft to `draft_output.txt`.

Scope and learning note

The deterministic substitutions above are deliberate: no model credentials, interactive input, embeddings, or vector database are needed. For that reason, the final states do not exactly reproduce LLM-generated replies from the video. The source project and scenario log record this difference. The course's hand-type-without-AI practice requirement remains a personal learning task; reviewing and manually retyping the basic syntax is needed before marking that checklist item complete.